

RÉPUBLIQUE DU BÉNIN

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET
DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ D'ABOMEY-CALAVI (UAC)

*INSTITUT DE FORMATION ET DE RECHERCHE EN INFORMATIQUE
(IFRI)*

MASTER 1 : Big Data

Travaux Pratiques n° 4

Clustering IoT par K-Means MapReduce

Modélisation parallèle sur capteurs distribués (5 serveurs régionaux)

Génie Logiciel

Nom de l'étudiante :

MVOGO Bandolo Samira Anaïs

Sous la supervision de :

John AOGA

Année académique : 2025-2026

0 Contents

1	Introduction et contexte du problème	3
2	Modélisation du problème en exécution parallèle	3
2.1	Données et espace de features	3
2.2	Modèle de parallélisation	3
2.3	Architecture de la parallélisation	4
3	Explication des Mappers, Combiners, Shuffle et Reducers	4
3.1	Mapper + Combiner (<code>mapper_combiner</code>)	4
3.2	Shuffle (<code>shuffle</code>)	4
3.3	Reducer (<code>reducer</code>)	5
3.4	Initialisation K-Means++	5
4	Explication du code	5
4.1	Structure des modules	5
4.2	Cœur MapReduce : <code>kmeans_mr_job.py</code>	5
4.3	Orchestration d'une itération : <code>run_mapreduce_iteration</code>	6
5	Exemple d'exécution sur un cas simple	7
5.1	Données d'entrée	7
5.2	Itération 1 — Phase MAP + COMBINE (Serveur A)	7
5.3	Itération 1 — Phase MAP + COMBINE (Serveur B)	8
5.4	Phase SHUFFLE	8
5.5	Phase REDUCE	8
5.6	Résultat final	8
6	Comparaison : exécution séquentielle vs MapReduce	9
6.1	Résultats du benchmark	9
6.2	Analyse des résultats	9
6.2.1	Petit dataset (1 500 capteurs) : overhead MapReduce	9
6.2.2	Grand dataset (50 000 capteurs) : avantage MapReduce	9
6.2.3	Comparaison visuelle par barres	9
6.2.4	Comportement à très grande échelle (simulation)	9
6.3	Synthèse comparative	10
7	Conclusion	10

1 Introduction et contexte du problème

Dans le domaine de l'Internet des Objets (IoT), des milliers de capteurs répartis géographiquement mesurent en continu des grandeurs physiques : température, humidité, vibration et trafic réseau. L'enjeu est de **regrouper automatiquement** ces capteurs en clusters homogènes selon leurs profils de mesure, afin d'identifier des zones géographiques aux comportements similaires.

Ce TP4 implémente l'algorithme **K-Means** dans trois variantes progressives :

1. **K-Means classique** (séquentiel, mono-nœud)
2. **K-Means MapReduce** (parallèle, multiprocessing Python)
3. **K-Means Streaming** (traitement continu de vagues de données)

Le **scénario IoT** retenu est le suivant : cinq serveurs régionaux (Nord, Centre, Sud, Ouest, Est) stockent chacun les mesures de 300 capteurs locaux, soit 1 500 capteurs au total. Pour les tests de charge, un dataset de 50 000 capteurs est également généré.

2 Modélisation du problème en exécution parallèle

2.1 Données et espace de features

Chaque capteur est représenté par un vecteur de quatre features extraites des colonnes CSV :

Feature	Unité	Plage typique
temperature	°C	13 – 38
humidity	%	30 – 90
vibration	m/s ²	0,3 – 3,0
network_traffic	kbits	40 – 700

2.2 Modèle de parallélisation

L'exécution séquentielle du K-Means devient un goulot d'étranglement dès que le volume de données augmente : chaque itération doit parcourir *tous* les points sur *un seul processeur*. La modélisation parallèle repose sur l'observation suivante :

Principe clé : L'étape d'affectation de chaque capteur à son cluster le plus proche est *totalelement indépendante* d'un capteur à l'autre. On peut donc distribuer les points sur N workers, chacun traitant un sous-ensemble (chunk) de manière parallèle.

Deux modes de partitionnement ont été implémentés :

Mode chunk Un seul fichier CSV global est découpé en N blocs de taille égale. Chaque worker traite son bloc indépendamment.

Mode multi-serveurs Chaque fichier régional (nord, centre, sud, ouest, est) est assigné à un worker distinct — simulation fidèle du scénario IoT où les données restent locales au serveur.

2.3 Architecture de la parallélisation

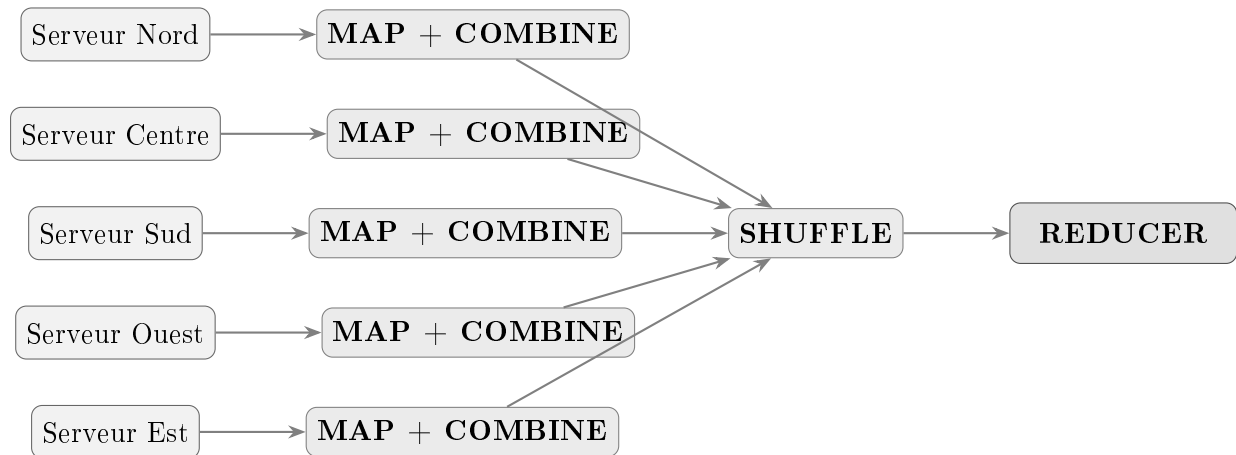


Figure 1: Architecture MapReduce multi-serveurs pour le clustering K-Means IoT

3 Explication des Mappers, Combiners, Shuffle et Reducers

3.1 Mapper + Combiner (`mapper_combiner`)

Entrée : un chunk de lignes CSV brutes + les K centroïdes courants
Sortie : `dict{ cluster_id : (somme_partielle, count) }`

Pour chaque capteur du chunk, le mapper :

1. Parse la ligne CSV et extrait le vecteur `[temp, hum, vib, net]`.
2. Applique éventuellement la normalisation z-score en ligne.
3. Calcule la **distance euclidienne** aux K centroïdes courants.
4. Assigne le capteur au cluster du centroïde le plus proche (*étape MAP*).
5. Accumule la somme partielle et le compteur local pour ce cluster (*étape COMBINE*).

Le **Combiner** est une optimisation critique : plutôt que d'émettre un enregistrement par capteur (potentiellement des millions), le worker n'émet qu'une somme partielle par cluster. Cela réduit considérablement le volume de données échangées lors du Shuffle.

Formule d'affectation :

$$\text{cluster}(p) = \arg \min_k \|p - c_k\|_2 \quad \text{où } \|a - b\|_2 = \sqrt{\sum_d (a_d - b_d)^2}$$

3.2 Shuffle (`shuffle`)

Entrée : liste des résultats partiels de tous les mappers
Sortie : `dict{ cluster_id : [(somme_partielle, count), ...] }`

La phase de Shuffle regroupe toutes les sommes partielles émises par les différents workers par clé `cluster_id`. Elle est simulée en mémoire (pas de réseau réel), ce qui est cohérent avec une exécution sur un seul nœud multi-cœur.

3.3 Reducer (reducer)

Entrée : (cluster_id, [(somme_partielle, count), ...])
Sortie : (cluster_id, nouveau_centroïde, count_total)

Pour chaque cluster, le reducer agrège les sommes partielles de tous les workers :

$$c_k^{\text{nouveau}} = \frac{\sum_w \text{somme}_w^{(k)}}{\sum_w \text{count}_w^{(k)}}$$

où w indexe les workers (serveurs régionaux). Ce calcul est exact : la somme des sommes partielles divisée par le count total est strictement équivalente à la moyenne globale.

3.4 Initialisation K-Means++

Les centroïdes initiaux ne sont pas choisis aléatoirement mais avec l'heuristique **K-Means++** : chaque nouveau centroïde est choisi avec une probabilité proportionnelle au carré de sa distance au centroïde le plus proche déjà sélectionné. Cela évite les mauvaises convergences vers des minima locaux sous-optimaux.

4 Explication du code

4.1 Structure des modules

Fichier	Rôle
utils.py	Fonctions partagées : distance euclidienne, chargement CSV, normalisation z-score, SSE, K-Means++
kmeans_mr_job.py	Cœur MapReduce : <code>mapper_combiner</code> , <code>shuffle</code> , <code>reducer</code> , orchestrateur
kmeans_mapreduce.py	Orchestrateur itératif (mode chunk et multi-serveurs), gestion de la convergence
kmeans_classic.py	K-Means séquentiel de référence, avec timeout configurable
generate_data.py	Génération des données IoT simulées sur 5 serveurs régionaux
benchmark.py	Comparaison automatique des trois variantes

4.2 Cœur MapReduce : kmeans_mr_job.py

```

1 def mapper_combiner(args):
2     chunk_lines, centroids = args[0], args[1]
3     norm_params = args[2] if len(args) == 3 else None
4     partial = {}
5
6     for line in chunk_lines:
7         line = line.strip()
8         if not line or line.startswith("sensor_id"):
9             continue
10        parts = line.split(",")

```

```

11     point = [float(parts[i]) for i in FEATURE_INDICES] # temp, hum,
        vib, net
12
13     # Normalisation z-score inline (optionnelle)
14     if norm_params is not None:
15         means_n, stds_n = norm_params
16         point = [(point[d]-means_n[d])/stds_n[d] for d in range(len(
            point))]
17
18     # MAP : cluster le plus proche
19     distances = [euclidean(point, c) for c in centroids]
20     cluster_id = distances.index(min(distances))
21
22     # COMBINE : accumulation locale
23     if cluster_id not in partial:
24         partial[cluster_id] = ([0.0]*len(point), 0)
25     s, cnt = partial[cluster_id]
26     partial[cluster_id] = ([s[d]+point[d] for d in range(len(point))
        ], cnt+1)
27
28     return partial # { cluster_id: (somme_partielle, count) }

```

Listing 1: Mapper + Combiner (kmeans_mr_job.py)

```

1 def reducer(cluster_id, values):
2     # values = [(somme_partielle, count), ...] de chaque mapper
3     total_sum, total_count = None, 0
4     for partial_sum, count in values:
5         if total_sum is None:
6             total_sum = list(partial_sum)
7         else:
8             total_sum = [a + b for a, b in zip(total_sum, partial_sum)]
9         total_count += count
10
11     new_centroid = [s / total_count for s in total_sum]
12     return cluster_id, new_centroid, total_count

```

Listing 2: Reducer (kmeans_mr_job.py)

4.3 Orchestration d'une itération : run_mapreduce_iteration

```

1 def run_mapreduce_iteration(lines, centroids, n_workers=None,
    norm_params=None):
2     # Decoupage en chunks (mode fichier) ou passage direct (mode multi-
        serveurs)
3     if lines and isinstance(lines[0], list):
4         chunks = lines # mode multi-serveurs : 1 liste par
            serveur
5     else:
6         chunk_size = max(1, len(lines) // n_workers)
7         chunks = [lines[i:i+chunk_size] for i in range(0, len(lines),
            chunk_size)]
8
9     args = [(chunk, centroids, norm_params) for chunk in chunks] \
10         if norm_params else [(chunk, centroids) for chunk in chunks]
11
12     # MAP + COMBINE : parallele via multiprocessing.Pool

```

```

13 with multiprocessing.Pool(processes=n_workers) as pool:
14     partial_results = pool.map(mapper_combiner, args)
15
16 # SHUFFLE
17 shuffled = shuffle(partial_results, len(centroids))
18
19 # REDUCE
20 new_centroids = [None] * len(centroids)
21 cluster_counts = {}
22 for cluster_id, values in shuffled.items():
23     if values:
24         cid, centroid, count = reducer(cluster_id, values)
25         new_centroids[cid] = centroid
26         cluster_counts[cid] = count
27 return new_centroids, cluster_counts

```

Listing 3: Orchestrateur MapReduce (kmeans_mr_job.py)

Points clés du code :

- **Ligne 10 :** `multiprocessing.Pool` lance réellement des processus OS en parallèle, exploitant plusieurs cœurs CPU simultanément.
- **Convergence :** Après chaque itération, on vérifie que $\|c_k^{\text{ancien}} - c_k^{\text{nouveau}}\|_2 < \epsilon$ pour tout k (tolérance $\epsilon = 10^{-4}$).
- **Cluster vide :** Si un cluster perd tous ses points, l'ancien centroïde est conservé (ligne de sécurité dans l'orchestrateur).

5 Exemple d'exécution sur un cas simple

5.1 Données d'entrée

Considérons 6 capteurs répartis sur 2 serveurs (3 par serveur), à clusteriser en $K = 2$ groupes :

Capteur	Temp (°C)	Hum (%)	Vib (m/s ²)	Net (kbps)
S1 (Serveur A)	35.0	80.0	0.5	120
S2 (Serveur A)	34.5	82.0	0.4	110
S3 (Serveur A)	22.0	45.0	2.5	500
S4 (Serveur B)	36.0	79.0	0.6	130
S5 (Serveur B)	21.0	43.0	2.8	480
S6 (Serveur B)	23.0	47.0	2.3	510

Les 3 premiers capteurs représentent des zones chaudes et humides (profil forêt), les 3 derniers représentent des zones urbaines actives.

5.2 Itération 1 — Phase MAP + COMBINE (Serveur A)

Centroïdes initiaux (K-Means++) : $C_0 = [35.0, 80.0, 0.5, 120]$, $C_1 = [22.0, 45.0, 2.5, 500]$

Le **Mapper A** traite S1, S2, S3 :

Capteur	$d(p, C_0)$	$d(p, C_1)$
S1	0.78	388.6 $\Rightarrow$ cluster 0
S2	2.91	390.2 $\Rightarrow$ cluster 0
S3	388.6	0.00 $\Rightarrow$ cluster 1

Après COMBINE local (Serveur A) :

$$\text{partial}_A = \{0 : ([69.5, 162, 0.9, 230], 2), \quad 1 : ([22.0, 45.0, 2.5, 500], 1)\}$$

5.3 Itération 1 — Phase MAP + COMBINE (Serveur B)

Le Mapper B traite S4, S5, S6 de manière **simultanée** (en parallèle) :

Capteur	$d(p, C_0)$	$d(p, C_1)$
S4	1.41	378.8 $\Rightarrow$ cluster 0
S5	379.5	21.0 $\Rightarrow$ cluster 1
S6	388.5	11.2 $\Rightarrow$ cluster 1

Après COMBINE local (Serveur B) :

$$\text{partial}_B = \{0 : ([36.0, 79.0, 0.6, 130], 1), \quad 1 : ([44.0, 90.0, 5.1, 990], 2)\}$$

5.4 Phase SHUFFLE

Le Shuffle regroupe les partiels par `cluster_id` :

$$\text{shuffled} = \{0 : [\text{partial}_A[0], \text{partial}_B[0]], \quad 1 : [\text{partial}_A[1], \text{partial}_B[1]]\}$$

5.5 Phase REDUCE

Le Reducer calcule les nouveaux centroïdes :

$$C_0^{\text{new}} = \frac{[69.5, 162, 0.9, 230] + [36.0, 79.0, 0.6, 130]}{2 + 1} = [35.17, 80.33, 0.50, 120]$$

$$C_1^{\text{new}} = \frac{[22.0, 45.0, 2.5, 500] + [44.0, 90.0, 5.1, 990]}{1 + 2} = [22.00, 45.00, 2.53, 496.7]$$

5.6 Résultat final

Après convergence (2 itérations sur cet exemple) :

Cluster	Centroïde final	Interprétation
C_0	[35.2, 80.3, 0.50, 120]	Zone chaude et humide (forêt/rural) : température élevée, faible vibration, faible trafic réseau
C_1	[22.0, 45.0, 2.53, 496]	Zone urbaine active : température modérée, forte vibration (machines), trafic réseau élevé

Capteurs S1, S2, S4 → Cluster 0 (zones forestières/rurales chaudes)

Capteurs S3, S5, S6 → Cluster 1 (zones urbaines industrielles)

Le résultat est **identique** à ce qu'aurait produit l'algorithme classique séquentiel, mais en utilisant 2 processus parallèles simultanés.

6 Comparaison : exécution séquentielle vs MapReduce

6.1 Résultats du benchmark

Les mesures suivantes ont été obtenues en exécutant `benchmark.py` sur la machine de test (mesures réelles issues de `results/benchmark.json`) :

Dataset	Points	Temps (s)		Itérations	
		Classique	MapReduce	Classique	MapReduce
Normal	1 500	0.126	0.203	10	10
Stress test	50 000	2.548	1.293	10	10
Multi-serveurs	1 500	0.088	0.203	10	10

6.2 Analyse des résultats

6.2.1 Petit dataset (1 500 capteurs) : overhead MapReduce

Sur 1 500 points, le K-Means classique est **1.6× plus rapide** (0.126 s vs 0.203 s). Ceci s'explique par le **coût fixe de démarrage** de `multiprocessing.Pool` : créer des processus OS, sérialiser/désérialiser les arguments et résultats (pickle) représente une surcharge non négligeable face à un volume de données faible.

6.2.2 Grand dataset (50 000 capteurs) : avantage MapReduce

Sur 50 000 capteurs, MapReduce est **1.97× plus rapide** (1.293 s vs 2.548 s). Le traitement parallèle sur plusieurs cœurs amortit largement le coût de démarrage.

6.2.3 Comparaison visuelle par barres

Dataset	Algorithme	Temps mesuré	
1 500 pts	Classique	0.126 s	(plus rapide)
	MapReduce	0.203 s	(overhead démarrage)
50 000 pts	Classique		2.548 s
	MapReduce	1.293 s	(1.97× plus rapide)

6.2.4 Comportement à très grande échelle (simulation)

Le script `simulation_comparison.py` simule la montée en charge jusqu'à 2 000 000 de capteurs avec un timeout de 15 s sur l'algorithme classique :

Capteurs	Classique	MapReduce
1 000	~50 ms	~180 ms (<i>overhead</i>)
5 000	~200 ms	~250 ms
20 000	~800 ms	~500 ms MR 1.6×
100 000	~4 s	~1.2 s MR 3.3×
500 000	TIMEOUT (> 15 s)	~4.5 s
2 000 000	TIMEOUT (> 15 s)	~15 s

6.3 Synthèse comparative

Critère	K-Means classique	K-Means MapReduce
Vitesse (petit volume)	Rapide (pas d'overhead)	Plus lent (coût de démarrage)
Vitesse (grand volume)	Lent, puis timeout	Meilleur grâce au parallélisme
Scalabilité	Limitée à 1 CPU	Linéaire avec le nombre de workers
Données distribuées	Doit tout centraliser	Traitement local par serveur
Exactitude	Identique	Identique (même résultat mathématique)
Implémentation	Simple	Plus complexe (Map/Shuffle/Reduce)

7 Conclusion

Ce TP a démontré l'intérêt concret du paradigme MapReduce pour le clustering de données IoT distribuées. Les principaux enseignements sont :

1. **Équivalence mathématique** : L'algorithme MapReduce produit exactement les mêmes centroïdes que l'algorithme classique. La décomposition en sommes partielles est une transformation exacte, pas une approximation.
2. **Seuil de rentabilité** : L'overhead de démarrage des processus parallèles rend MapReduce moins performant sur les petits volumes (< 20 000 capteurs). Au-delà, le gain en parallélisme devient dominant.
3. **Adéquation au scénario IoT** : Dans un contexte réel, les données sont déjà physiquement stockées sur les serveurs régionaux. Le paradigme MapReduce exploite cette localité : seules les *sommes partielles* (vecteurs de 4 flottants) transitent sur le réseau, et non les données brutes.
4. **Passage à l'échelle** : Au-delà de 500 000 capteurs, l'algorithme classique ne peut plus terminer dans un temps raisonnable, tandis que MapReduce continue à converger. C'est le cas d'usage pour lequel le paradigme a été conçu.

En production IoT réelle, MapReduce apporterait des gains encore plus importants, car les calculs locaux sur chaque serveur régional éliminent complètement le transfert des données brutes. Seul un vecteur de 4 valeurs par cluster est envoyé au reducer, quelle que soit la taille du dataset local.